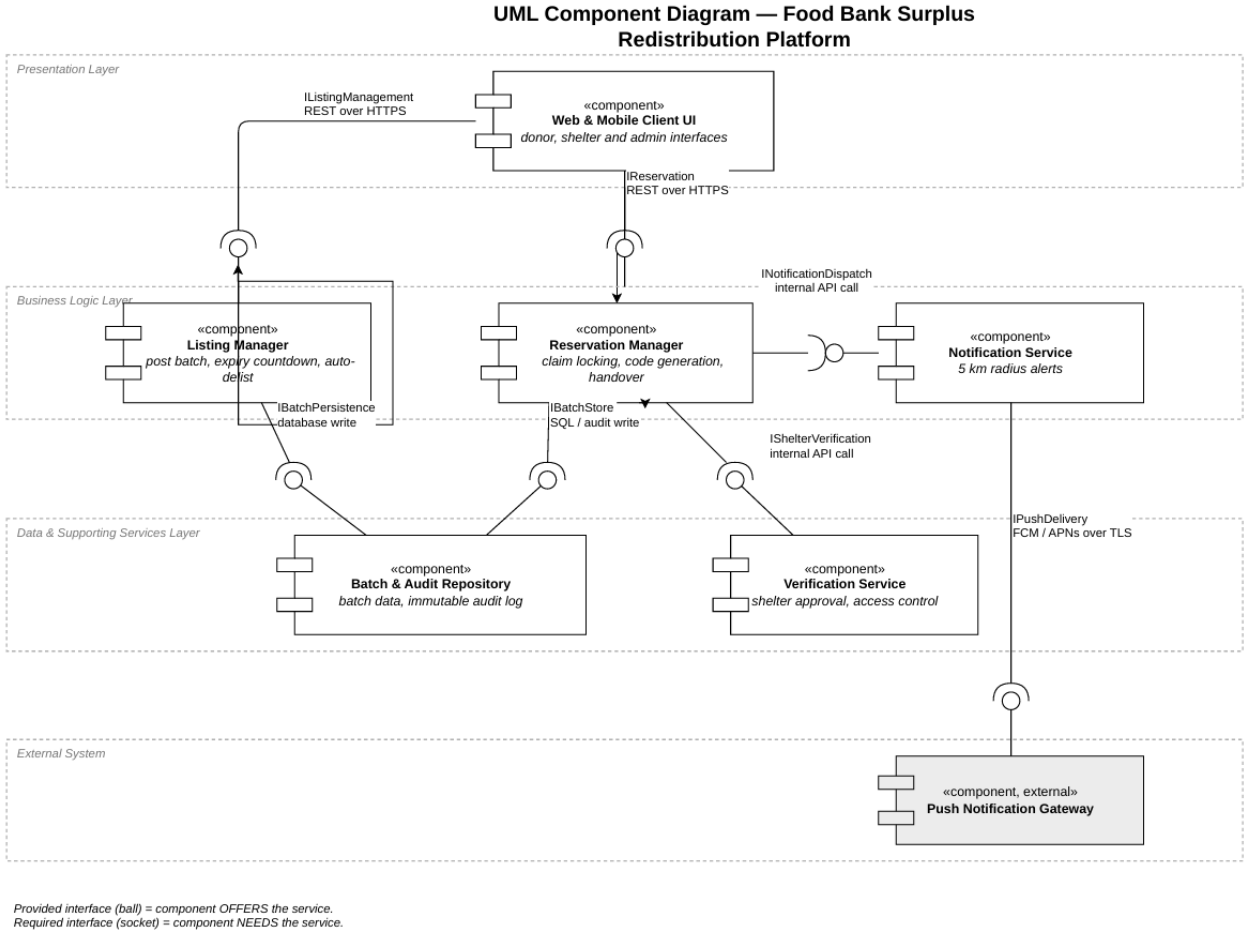
Lab 3: Architectural Justification

Food Bank Surplus Redistribution Platform (Problem Statement #62)

Name: Mythri S SRN: PES1UG24AM419 Section: G

We chose the Layered Architecture for the Food Bank Surplus Redistribution Platform.



Architectural Choice

The platform is divided into four layers. The Presentation Layer contains the Web and Mobile Client UI, which is used by donor partners, shelter coordinators, and administrators. The Business Logic Layer contains the Listing Manager, Reservation Manager, and Notification Service. The Data & Supporting Services Layer contains the Batch & Audit Repository and Verification Service. The Push Notification Gateway is an external system outside our platform. Each layer communicates only with the layer directly below it through defined interfaces. This makes the system easier to understand, maintain, and modify.

Reason 1 — Traceability is far simpler with a single transactional store

NFR-002 requires an audit record to be created for every listing, reservation, expiry, and collection event. These records must also remain unchanged after they are created. With the layered design, all these events are stored in the same Batch & Audit Repository and can be handled within one database transaction. This means the audit record and the actual state change happen together, so there is less chance of missing an audit entry. If we used separate microservices with different databases, keeping all these records consistent would be much more difficult and could require distributed transactions or eventual consistency. For a system that needs a reliable food-safety audit trail, using one database is much simpler and safer.

Reason 2 — The feature set is small, fixed and delivered by one team

The platform has only five main functional requirements, and there are no major differences in how the different parts of the system need to scale. Since the system is relatively small and will be developed by one team, a layered architecture fits the project well. The layers naturally separate the user interface, business operations, stored data, and notification system. Using

microservices would introduce extra work such as service discovery, separate deployment processes, and communication between different services. These features are useful for large systems, but they are not really necessary for this platform at its current size.

Security Advantage

The layered architecture also makes it easier to enforce the verified-shelter rule in NFR-002. The Reservation Manager is responsible for changing a batch to the RESERVED state, and before doing this, it checks the shelter's approval status through the Verification Service. The Presentation Layer cannot directly access the database or bypass this process. This means that even if a client sends incorrect or modified data, an unverified shelter will still be rejected. Keeping this security check in one place also makes the rule easier to maintain and audit.

Performance Benefit

FR-002 requires the system to filter batches based on distance, dietary tags, and remaining time, and return the results within two seconds for up to 500 active batches. With the layered design, this can be handled using a single indexed query on the Batch & Audit Repository. Since the query is handled within the same system, there is no need for multiple services to communicate over the network just to collect the required information. The only part that needs an external network call is the push notification required by NFR-001. This notification is handled asynchronously, so it does not make the shelter wait while browsing the claim board.

Trade-off Acknowledged

The Notification Service cannot be scaled separately in this design. If the platform grows significantly, it can be separated into its own service. For the current scale, the simpler layered design is more suitable.